Pak-Austria Fachhochschule

Institute of Applied Sciences and Technology

Skilling Pakistan

Submitted To:

Dr. Nabeel Ahmad Awan

Software Construction & Development Instructor

PAF-IAST

CCP Documentation

Department of IT & CS Discipline:

Software Engineering PAF-IAST

MAY 1, 2026

Team Members:

Names	***Registration No***
1. Hamza Badshah	B23F0352SE016
2. Hammad Tahir	B23F0649SE087
3. Hafsa Khan	B23F0580SE009
4. Aman Sajid	B23F0107SE081
5. Esha Chatta	B23F0403SE077

Project: Cyber Threat Intelligence Pipeline (CCP)

1. Introduction

In the modern digital landscape, security operations centers (SOCs) are overwhelmed by the sheer volume of cyber threats. To effectively mitigate these risks, organizations require automated systems capable of ingesting, analyzing, and visualizing threat indicators in real-time. This document outlines the complete software engineering lifecycle of the Cyber Threat Intelligence Pipeline, developed as a Complex Coursework Project (CCP).

The engineered solution is a highly scalable, event-driven microservices architecture built utilizing Java Spring Boot, Apache Kafka, and MySQL. It automatically aggregates raw Indicators of Compromise (IoCs) from leading global threat databases specifically AbuseIPDB and AlienVault OTX. Instead of a monolithic approach, the system relies on distributed processing, decoupled components, and a dedicated Ranking Service for data enrichment, ultimately serving actionable intelligence to a live, dark-themed SOC analytics dashboard.

This documentation spans the architectural blueprints, technical configurations, data modeling schemas, fault tolerance mechanisms, and the complete operational deployment strategy required to maintain and scale the system.

2. High-Level System Architecture

The architecture is fundamentally grounded in the Microservices design pattern, utilizing an Event-Driven methodology to ensure asynchronous, non-blocking communication between services. Apache Kafka acts as the central nervous system (message broker), decoupling the ingestion of data from the processing and storage of data.

2.1 Architectural Benefits

- **Scalability:** Each microservice can be scaled independently based on load. If ingestion spikes due to a global cyber-attack, only the Ingestion Service needs more resources.

- **Fault Tolerance:** If the database goes offline, the Ingestion Service continues to fetch data and store it safely in the Kafka queue until the Database/Extraction service comes back online.

- **Polyglot Capabilities:** The clear boundaries defined by REST APIs and Kafka topics allow future modules to be written in different languages (e.g., Python for Machine Learning analysis) without disrupting the Java backend.

3. Component-Level Technical Specifications

3.1 Microservice 1: Ingestion Service (Data Producer)

- **Port Designation:** 8080
- **Primary Role:** To act as the initial gateway for all incoming cyber threat data.

Technical Implementation:

The Ingestion Service relies on Spring Boot's RestTemplate to execute HTTP GET requests against external threat feeds. It manages two distinct integration modules:

1. AbuseIPDB Module: Queries the /api/v2/blacklist endpoint to retrieve IPs with a confidence score of 90% or higher.

2. AlienVault OTX Module: Connects to the /api/v1/pulses/subscribed endpoint to retrieve complex threat pulses and associated indicators.

Once the raw JSON payload is successfully retrieved, the service utilizes a KafkaTemplate to publish the unaltered payload to the "raw-iocs" Kafka topic. It is intentionally "dumb" regarding the content of the data, focusing solely on reliable fetch-and-publish mechanics.

3.2 Microservice 2: Extraction & Processing Service

- **Port Designation:** 8082
- **Primary Role:** Data consumption, validation, enrichment, and persistence.

Technical Implementation:

This service houses a @KafkaListener bound to the "raw-iocs" topic. When a message arrives, it executes a complex orchestration pipeline:

- **Payload Identification:** Utilizing Jackson ObjectMapper, it inspects the JSON structure to determine if the origin was AbuseIPDB (array under "data") or AlienVault (array under "results").

- **Validation:** Null, empty, or malformed indicators are immediately discarded to maintain database integrity.

- **Enrichment (Ranking Service**): Before saving, the indicator is passed to an internal RankingService module. This simulates querying an external validation tool (like VirusTotal) to generate an independent Enriched Severity Score based on a deterministic hash algorithm.

- **Persistence:** The finalized, enriched threat object is mapped via Hibernate/JPA and saved to the MySQL relational database.

3.3 Microservice 3: Threat API Service

- **Port Designation:** 8083
- **Primary Role:** Serving processed data to external presentation layers.

Technical Implementation:

- This is a lightweight Spring Boot application containing a @RestController. It securely connects to the MySQL database via Spring Data JPA and exposes a @GetMapping at "/api/threats".

- Crucially, it implements Cross-Origin Resource Sharing (CORS) via the @CrossOrigin annotation, permitting browser-based frontend applications on different ports to securely request the JSON payload.

4. Data Modeling and Schema

The relational database is orchestrated by WAMP server (MySQL port 3306). The primary table `iocs` (Indicators of Compromise) is automatically generated by Hibernate and follows this schema:

- **id** (Long, Primary Key, Auto-Increment)
- **ioc_value (String, Varchar) -** The malicious IP or Domain.
- **ioc_type (String, Varchar) -** E.g., "IP" or "Indicator".
- **severity_score (Integer) -** The enriched threat level (0-100).
- **source (String) -** The originating API (AbuseIPDB or AlienVault).
- **created_at (Timestamp) -** Handled automatically by the database for temporal tracking.

5. Presentation Layer: SOC Web Dashboard

The frontend abandons heavy JavaScript frameworks in favor of a highly optimized, single-file HTML/CSS/Vanilla JS approach, styled natively as a dark-mode Security Operations Center (SOC) terminal.

Key Features:

- **Asynchronous Fetch API:** Utilizes async/await to continuously poll the Threat API Service without blocking the UI thread.

- **Real-Time Analytics Engine:** The JavaScript layer calculates complex Key Performance Indicators (KPIs) on the fly, including Total Threats, Critical Volume (Score > 80), and Source Distribution.

- **DOM Rendering:** Dynamically constructs an HTML table of the top 100 most recent threats, utilizing conditional CSS formatting (e.g., highlighting critical scores in neon red) to alert human operators to immediate dangers.

6. Security & Fault Tolerance Measures

To meet enterprise standards, several layers of resilience were engineered into the pipeline:

6.1 API Key Management

Hardcoded credentials are strictly prohibited. External API keys for AbuseIPDB and AlienVault are isolated within the application.properties files and injected at runtime via Spring's @Value annotation. This ensures keys remain secure and easily rotatable.

6.2 Exception Handling

The pipeline implements robust Try Catch blocks at critical failure points. If an external API timeout occurs, the RestTemplate captures the exception, logs it via System.err, and prevents the Spring Boot instance from crashing. Similarly, if the Kafka Listener encounters an unparseable JSON string, it safely discards the anomalous message rather than halting the consumption queue.

7. Deployment & Operations Guide

To ensure a stable local environment, the execution sequence must strictly adhere to the Dependency Boot Order:

- **Step 1 (Infrastructure):** Launch WAMP Server to initialize the MySQL database. Concurrently, launch the Apache Kafka server via PowerShell (kafka-server-start.bat).

- **Step 2 (Consumers):** Execute the Extraction Service (Port 8082) to establish the Kafka listener, followed by the Threat API Service (Port 8083) to open web traffic.

- **Step 3 (UI):** Open index.html to establish the analytics dashboard.

- **Step 4 (Producer):** Execute the Ingestion Service (Port 8080) and hit the /ingest endpoint to trigger the end-to-end data flow.

8. Future Scope and Scalability

While the current iteration fully satisfies the requirements of the CCP, the architecture is designed to support significant future expansion. Recommended next steps for a production-grade deployment include:

- **Containerization (Docker):** Wrapping each microservice, the database, and the Kafka broker into Docker containers orchestrated by Docker Compose or Kubernetes. This would eliminate the manual startup sequence and "works on my machine" inconsistencies.

- **ELK Stack Integration:** Funneling the Threat API data into Elasticsearch, Logstash, and Kibana to provide historical search capabilities and massively expanded data visualization beyond the current 100-row limit.

- **Machine Learning Anomaly Detection:** Implementing a Python-based Kafka consumer to run statistical models against the incoming IP addresses, attempting to predict malicious behavior before it is officially blacklisted by AlienVault or AbuseIPDB.

9. Conclusion

The Cyber Threat Intelligence Pipeline represents a comprehensive application of advanced software engineering methodologies. By successfully decoupling the ingestion, processing, and visualization layers via Apache Kafka and Spring Boot, the project achieves a high degree of fault tolerance and architectural clarity.

The integration of multiple external APIs, coupled with a dedicated enrichment ranking algorithm, ensures that the data served to the SOC dashboard is both actionable and highly accurate.

The resulting application not only meets all academic requirements outlined for the Complex Coursework Project but also serves as a robust foundation for an enterprise-grade cybersecurity monitoring solution.